

LABORATORIO DI SISTEMI OPERATIVI

Pipe

fabio.rossi@unipg.it

075 5855005

IPC (interprocess communication)

- vari strumenti per la comunicazione tra processi:
 - signal
 - socket
 - **pipe**
 - shared memory
 - memory-mapped file
 - etc.

Pipe

- unidirezionali (in *GNU/Linux*. In certi sistemi sono bidirezionali. POSIX.1 definisce solo pipe unidirezionali)
- uno dei primi meccanismi di *IPC* e sono disponibili praticamente su tutti i sistemi
- le abbiamo viste anche nella shell:
`cat pippo.txt | sort | less`
- connettono due o più processi tramite un canale di comunicazione
- hanno una "estremità" per la lettura (*read end*) ed una per la scrittura (*write end*)
- comunicazione non strutturata (*byte stream*)

Pipe

- due tipi:
 1. senza nome (*unnamed*)
 2. con nome (*named* o *FIFO*)
- **Unnamed:** sono create con *pipe()*, i processi comunicanti condividono file descriptor per la lettura e la scrittura (*fork(2)*). Solo tra processi relati.
- **Named:** sono un file speciale (type *p* in *ls -l*) visibile nel file system (ma non occupano spazio né implicano operazioni in esso)
- una volta create si usano allo stesso modo:

"The only difference between pipes and FIFOs is the manner in which they are created and opened. Once these tasks have been accomplished, I/O on pipes and FIFOs has exactly the same semantics" (***man 7 pipe***)

(unnamed) Pipe

- solo processi relati/discendenti (*fork(2)*)
- i *file descriptor* per le due estremità (di lettura e di scrittura) sono duplicati dalla *fork(2)* e quindi disponibili anche nel *child*

Named Pipe o FIFO

- sono visibili nel filesystem ma il "nome" nel filesystem è solo un riferimento. Non occupano spazio disco né implicano operazioni in esso (i dati transitano nel *kernel*)
- da **man 7 pipe**: "*although FIFOs have a pathname in the filesystem, I/O on FIFOs does not involve operations on the underlying device (if there is one)*"
- possono essere trattate come qualsiasi file (SC per l'I/O)
- visibili nel sistema come gli altri file
 - qualsiasi processo può fare la open (se ne ha il permesso...)
 - possiamo impostare permessi per gli utenti
 - possiamo rimuoverle con *rm*

Man per le Pipe

- ***man 7 pipe*** per una panoramica (overview) sulle *pipe*
- ***man 7 fifo*** per una panoramica sulle *named pipe*
- ***man 2 pipe*** per il manuale della *system calls pipe(2)* che crea una *unnamed pipe* (solo tra processi "imparentati" con *fork(2)*)
- ***man 3 mkfifo*** per il manuale della funzione di libreria C (*wrapper*) che crea una *named pipe* (visibile nel *filesystem*)
- ***man 1 mkfifo*** per il manuale sul comando *unix* che crea una *named pipe* (visibile nel *filesystem*) nella shell

Bloccaggio delle Pipe

- le operazioni di apertura/lettura/scrittura possono essere bloccanti o non bloccanti, cioè ritornare al chiamante solo dopo che si sia verificato un certo evento oppure ritornare subito
- dipende da:
 - la condizione della *pipe*, per esempio se vi sono stati scritti dati, se dall'altro lato della *pipe* qualcuno legge/scrive, se è piena o c'è ancora spazio, etc.
 - sia stato settato per la *pipe* il flag **O_NONBLOCK** (vedi anche *open(2)*)
- non sono possibili operazioni di *seek* (no *lseek(2)*)

Bloccaggio delle Pipe

se il flag **O_NONBLOCK** non è settato (di default non lo è) le operazioni sono bloccanti nei seguenti casi:

- ***open(2)***: si blocca se dall'altro lato nessun processo ha aperto la pipe nella modalità di accesso opposta (*read/write*)
- ***read(2)***: si blocca se la pipe è vuota (non ci sono dati)
- ***write(2)***: si blocca se la pipe è piena

Creazione di una pipe unnamed

```
int pipefd[2];  
if (pipe(pipefd) == -1) {  
    perror("pipe");  
    exit(EXIT_FAILURE);  
}
```

- **pipefd[0]** file descriptor per la *read end* della pipe
- **pipefd[1]** file descriptor per la *write end* della pipe
- **Ritorno : 0** se tutto ok. **-1** per errore

`int pipe(int pipefd[2])`

- Crea la pipe e apre le due estremità
- creata la pipe è possibile leggere e scrivere dati, per es.: con **read()** e **write()**
- Alla creazione di default il flag **O_NONBLOCK** non è settato, quindi sia lettura che scrittura possono bloccarsi.

(unnamed) Pipe

- sequenza tipica di utilizzo:
 - il parent crea la pipe (*pipe()*)
 - il parent crea un child (*fork()*). I file descriptor ritornati dalla *pipe()* sono duplicati nel child
 - lo scrivente chiude il *fd* di lettura*
 - il lettore chiude il *fd* di scrittura*
 - i processi comunicano con *read()* e *write()*
 - al termine entrambi chiudono il *fd* utilizzato

(*): questo per avere un comportamento "corretto" nel caso che dall'altra lato tutti i processi abbiano chiuso la pipe

Creazione di una named pipe

- `int mkfifo(const char *pathname,
mode_t mode)`
- crea una *named pipe* di nome *pathname*
- visibile nel filesystem ma non occupa spazio
- *mode* specifica i permessi, in accordo con *umask* ($mode \& \sim umask$)
- utilizzabile da qualsiasi processo (se ha il permesso...)

Apertura di una named pipe

```
int open(const char *pathname, int flags);
```

- una volta creata, la *named pipe* deve essere aperta
- nel caso in cui il lato opposto non sia ancora aperto **open** è bloccante (a meno che sia settato il flag **O_NONBLOCK**)
- in *Linux* è possibile aprire una named pipe in lettura e scrittura. Ciò permette di raggiungere certi scopi ma va usato con cautela.

Da *man 7 fifo*: "*Under Linux, opening a FIFO for read and write will succeed both in blocking and nonblocking mode. POSIX leaves this behavior undefined*"

ndr. in passato è stato usato da studenti nel progetto ma senza troppa consapevolezza! (ed ha generato problemi)

Capacità delle pipe

- le pipe hanno una capacità limitata, che varia a seconda dell'implementazione
- in Linux di default attualmente è 16 "*system page size*" (che dipende dal sistema), ma è configurabile (da 2.6.35) con *fcntl(2)*

(*man 7 pipe*)

Pipe: scrittura

- **ret = write(fdw, buffer, size)**
 - i dati sono scritti nella *pipe* sequenzialmente
 - se la pipe è piena ulteriori *write(2)* si bloccheranno (o falliranno se **O_NONBLOCK** è settato)
 - POSIX stabilisce che le *write(2)* di ampiezza minore di **PIPE_BUF** sono garantite essere atomiche (ma possono essere bloccanti se la pipe è piena)
 - Stabilisce anche che **PIPE_BUF** deve essere almeno 512 byte
 - vedi *man 7 pipe* per maggiori dettagli

Pipe: scrittura

- `ret = write(fdw, buffer, size)`
- se tutti i descrittori di lettura sono stati chiusi una *write* provoca l'invio di un segnale **SIGPIPE** al processo:
 - ✓ se SIGPIPE è gestito da un handler o ignorato: *write* ritorna -1; ***errno*** settato a **EPIPE**
 - ✓ se SIGPIPE non è gestito: la gestione di default provoca la terminazione del processo (!)

Pipe: lettura

- **ret = read(fdr, buffer, size)**
 - i dati sono letti in ordine di arrivo
 - i dati letti sono tolti dalla *pipe*
 - **O_NONBLOCK:**
 - ✓ **se non settato** (generalmente il default) :
 - ❖ se la pipe è vuota e i descrittori di scrittura non sono ancora stati chiusi la read si blocca in attesa
 - ❖ se i descrittori di scrittura sono tutti chiusi ritorna subito 0 (end_of_file)
 - ❖ altrimenti legge al più *size* byte, se al momento dell'invocazione la *pipe* contiene meno di *size* byte li legge tutti e ritorna il numero dei byte letti *ret*
 - ✓ **se settato:**
 - ❖ se la *pipe* è vuota la *read* ritorna subito -1; *errno EAGAIN*

Pipe: close()

- **ret = close(fd)**
 - quando l'ultimo descrittore di scrittura è chiuso genera *EOF* per i lettori (facendo ritornare 0 ad eventuali read in attesa)
 - se viene effettuata una *write* su una *pipe* in cui tutti i descrittori di lettura sono stati chiusi il processo riceve *SIGPIPE*

Named Pipe da shell

- comando *mkfifo(1)*
- crea un file speciale di tipo **p**
- ci possiamo scrivere e leggere da shell con i comandi usuali per i file
- es.:
 - **mkfifo mypipe**
 - **echo "Ciao" > mypipe**
 - **cat mypipe**